Introduction to Statistical Learning and Application - Master of Applied Mathematics

Razan MHANNA

STATIFY team, Inria, University Grenoble Alpes
Team 5, Grenoble Institute of Neurosciences GIN

March 19, 2026

Outline

1. Model Selection
   - Subset Selection
2. Cross-validation
   - K-Fold Cross-Validation
   - Leave-One-Out Cross-Validation (LOOCV)
   - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

Model selection

Three classes of methods:

- Subset Selection by taking a subset of the p predictors, then fit a model using least squarres on the reduced set of variables.
- Shrinkage: All p predictors are involved, but the estimated coefficients are shrinken towards zero relative to the least squares estimates. The shrinkage (also known as regularization) has the effect of reducing variance and can also perform variable selection.
- Dimension Reduction: Project the p predictots into a M-dimensional subspace, where M < p. This is achieved by computing M different linear combinations, or projections, of the variables. Then these M projections are used as predictors to fit a linear regression model by least squarres.

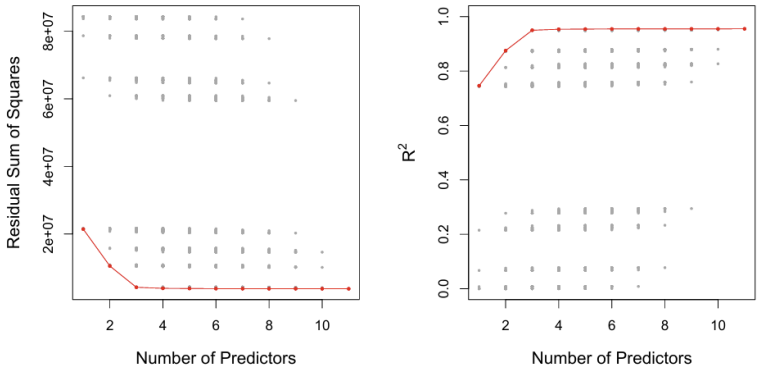
Outline

1. Model Selection
   - Subset Selection
2. Cross-validation
   - K-Fold Cross-Validation
   - Leave-One-Out Cross-Validation (LOOCV)
   - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

Best Subset Selection

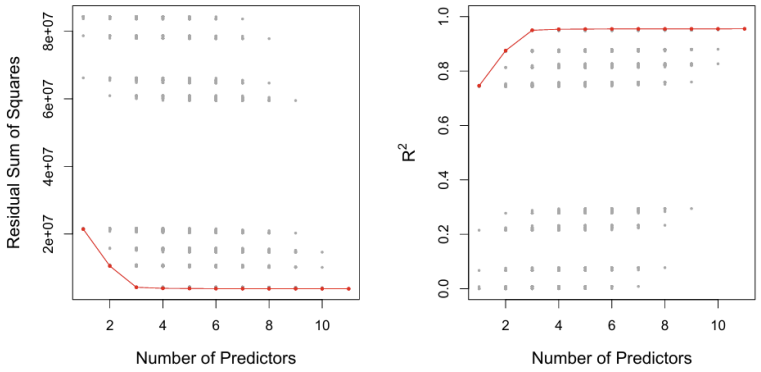
1. Let $\mathscr{M}_0$ denote the *null model*, which contains no predictors. This model simply predicts the sample mean for each observation.
2. For $k = 1, 2, \ldots, p$:
   1. Fit all $\binom{p}{k}$ models that contain exactly k predictors.
   2. Pick the best among these $\binom{p}{k}$ models, and call it $\mathscr{M}_k$. Here, *best* is defined as having the smallest RSS, or equivalently the largest R^2.
3. Select a single best model from among $\mathscr{M}_0, \ldots, \mathscr{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2.

Example dataset



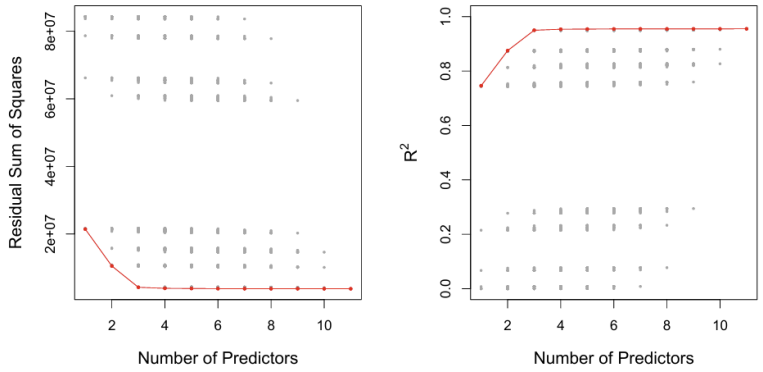
- Each point on the plot represents a least squares regression model built from a different subset of predictors in the Credit data.
- In total, 11 variables are considered (10 predictors plus two dummy variables for the qualitative variable region).
- The graph plots the RSS and R^2 as functions of the number of predictors, with red curves linking the best models.
- The results show that while model performance improves with more predictors, additional predictors yield only

Example dataset



- Each point on the plot represents a least squares regression model built from a different subset of predictors in the Credit data.
- In total, 11 variables are considered (10 predictors plus two dummy variables for the qualitative variable region).
- The graph plots the RSS and R^2 as functions of the number of predictors, with red curves linking the best models.
- The results show that while model performance improves with more predictors, additional predictors yield only

Example dataset



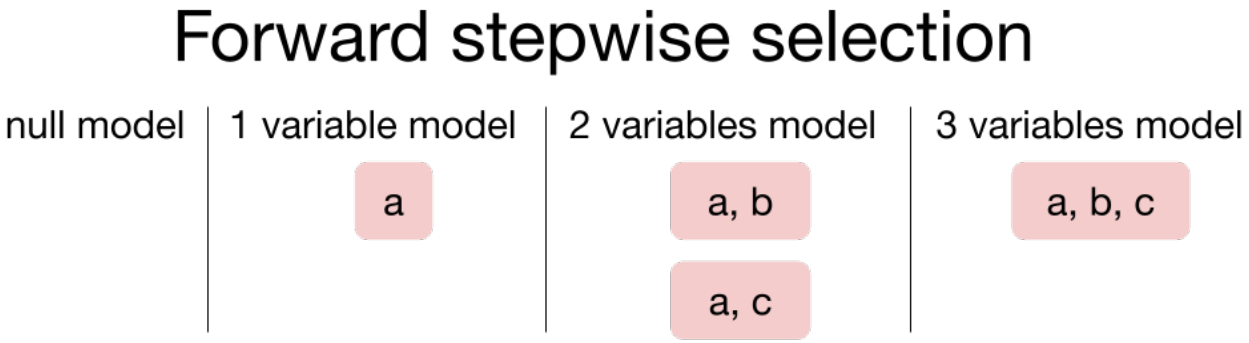
- Each point on the plot represents a least squares regression model built from a different subset of predictors in the Credit data.
- In total, 11 variables are considered (10 predictors plus two dummy variables for the qualitative variable region).
- The graph plots the RSS and R^2 as functions of the number of predictors, with red curves linking the best models.
- The results show that while model performance improves with more predictors, additional predictors yield only

Limitations of Best Subset Selection

- For computational reasons, best subset selection cannot be applied with very large p. *Why not?*
- Best subset selection may also suffer from statistical problems when p is large: larger the search space, the higher the chance of finding models that look good on the training data, even though they might not have any predictive power on future data.
- Thus an enormous search space can lead to *overfitting* and high variance of the coefficient estimates.
- For both of these reasons, *stepwise* methods, which explore a far more restricted set of models, are attractive alternatives to best subset selection.

Forward Stepwise Selection

- Forward stepwise selection begins with a model containing no predictors, and then adds predictors to the model, one-at-a-time, until all of the predictors are in the model.
- In particular, at each step the variable that gives the greatest *additional* improvement to the fit is added to the model.



Forward Stepwise Selection

Forward Stepwise Selection

1. Let $\mathscr{M}_0$ denote the *null* model, which contains no predictors.
2. For $k = 0, \ldots, p-1$:
   1. Consider all $p-k$ models that augment the predictors in $\mathscr{M}_k$ with one additional predictor.
   2. Choose the *best* among these $p-k$ models, and call it $\mathscr{M}_{k+1}$. Here *best* is defined as having smallest RSS or highest R^2.
3. Select a single best model from among $\mathscr{M}_0, \ldots, \mathscr{M}_p$ using cross-validated prediction error, C_p (AIC), BIC, or adjusted R^2.

More on Forward Stepwise Selection

- Computational advantage is clear.
- It is not guaranteed to find the best possible model out of all 2^p models containing subsets of the p predictors. *Why*

Concerns about model selection

- Given several candidate models, we can use **cross validation** to compare their predictive performance. But does it always make sense to choose the model that optimizes estimated predicted performance?
  1. If the only goal is prediction on data similar to what was observed, it can be better to average across models rather than to choose just one.
  2. If the goal is prediction on a new set of data, we should reweight the predictive errors accordingly to account for differences between sample and population,
  3. Rather than choosing among or averaging over an existing set of models, a better choice can be to perform continuous model expansion, constructing a larger model that encompasses the earlier models as special cases.

Outline

1. Model Selection
2. Cross-validation
   - Definition
   - Types of cross-validation
     - K-Fold Cross-Validation
     - Leave-One-Out Cross-Validation (LOOCV)
     - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

Outline

1. Model Selection
2. Cross-validation
   - Definition
   - Types of cross-validation
     - K-Fold Cross-Validation
     - Leave-One-Out Cross-Validation (LOOCV)
     - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

Cross-Validation

Definition

Cross-validation is a statistical method used to estimate the performance (or accuracy) of predictive statistical models.

- In cross-validation, you make a fixed number of folds (or partitions) of the data, run the analysis on each fold, and then average the overall error estimate.

[1] Brooke Lampe and Weizhi Meng.
can-train-and-test: A curated can dataset for automotive intrusion detection.
arXiv preprint arXiv:2308.04972, 2023.

[2] Li Yang, Abdallah Shami, Gary Stevens, and Stephen De Rusett.
LCCDE: A decision-based ensemble framework for intrusion detection in the internet of vehicles

Outline

1. Model Selection
2. Cross-validation
   - Definition
   - Types of cross-validation
     - K-Fold Cross-Validation
     - Leave-One-Out Cross-Validation (LOOCV)
     - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

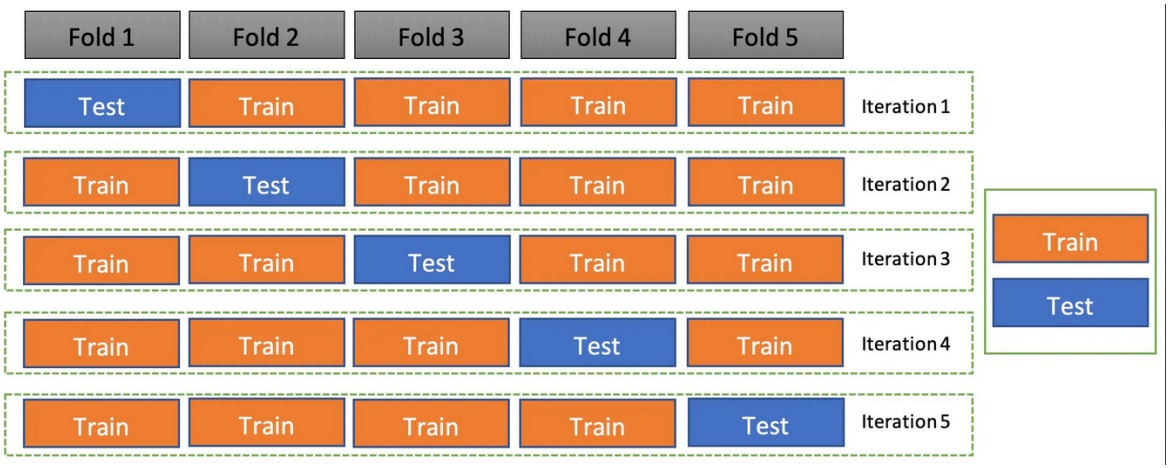
Types of Cross Validation Methods

There are different types of cross-validation methods, and they could be classified into two broad categories:

- **Non-exhaustive**
  - Holdout
  - K-Fold
  - Stratified K-Fold
- **Exhaustive Methods**
  - Leave-P-Out cross-validation
  - Leave-1-Out cross-validation

K-Fold Cross-Validation

1. Randomly split your dataset into k subsets (folds).
2. For each fold:
   - Train the model using $k-1$ folds.
   - Test the model on the remaining fold.
3. Repeat until each fold has been used as a test set.
4. Compute the average accuracy across all folds.



Advantages of k-Fold Cross-Validation

- Provides a reliable estimate of generalization error
- Reduces variance compared to a single train/test split
- Efficient use of data: each observation is used for both training and validation
- Less biased than hold-out validation (larger effective training sets)
- Naturally suited for model selection and hyperparameter tuning
- Allows analysis of stability of performance across folds
- Particularly useful for small to moderate sample sizes

Advantages of k-Fold Cross-Validation

- Provides a reliable estimate of generalization error
- Reduces variance compared to a single train/test split
- Efficient use of data: each observation is used for both training and validation
- Less biased than hold-out validation (larger effective training sets)
- Naturally suited for model selection and hyperparameter tuning
- Allows analysis of stability of performance across folds
- Particularly useful for small to moderate sample sizes

Advantages of k-Fold Cross-Validation

- Provides a reliable estimate of generalization error
- Reduces variance compared to a single train/test split
- Efficient use of data: each observation is used for both training and validation
- Less biased than hold-out validation (larger effective training sets)
- Naturally suited for model selection and hyperparameter tuning
- Allows analysis of stability of performance across folds
- Particularly useful for small to moderate sample sizes

Advantages of k-Fold Cross-Validation

- Provides a reliable estimate of generalization error
- Reduces variance compared to a single train/test split
- Efficient use of data: each observation is used for both training and validation
- Less biased than hold-out validation (larger effective training sets)
- Naturally suited for model selection and hyperparameter tuning
- Allows analysis of stability of performance across folds
- Particularly useful for small to moderate sample sizes

Limitations

- Computational cost: requires fitting the model K times
- Expensive for complex models or large datasets
- CV estimates are not independent (overlapping training sets)
- Can lead to selection bias when used for model tuning
- Sensitive to the choice of K (bias–variance trade-off)
- Standard K-Fold is not appropriate for time-dependent data

Limitations

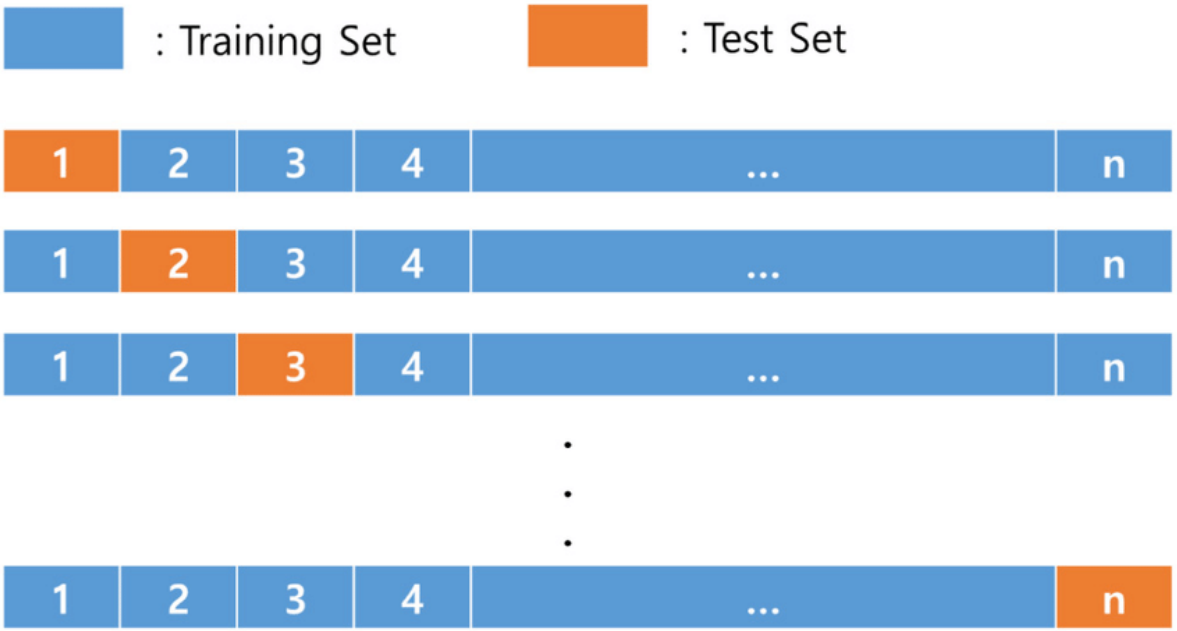
- Computational cost: requires fitting the model K times
- Expensive for complex models or large datasets
- CV estimates are not independent (overlapping training sets)
- Can lead to selection bias when used for model tuning
- Sensitive to the choice of K (bias–variance trade-off)
- Standard K-Fold is not appropriate for time-dependent data

Limitations

- Computational cost: requires fitting the model K times
- Expensive for complex models or large datasets
- CV estimates are not independent (overlapping training sets)
- Can lead to selection bias when used for model tuning
- Sensitive to the choice of K (bias–variance trade-off)
- Standard K-Fold is not appropriate for time-dependent data

Leave-One-Out Cross-Validation (LOOCV)

- LOOCV is a cross-validation technique where a single data point is used as the validation set, and the rest of the data is used for training.
- This process is repeated for each data point, ensuring that each observation is used once as the validation data.

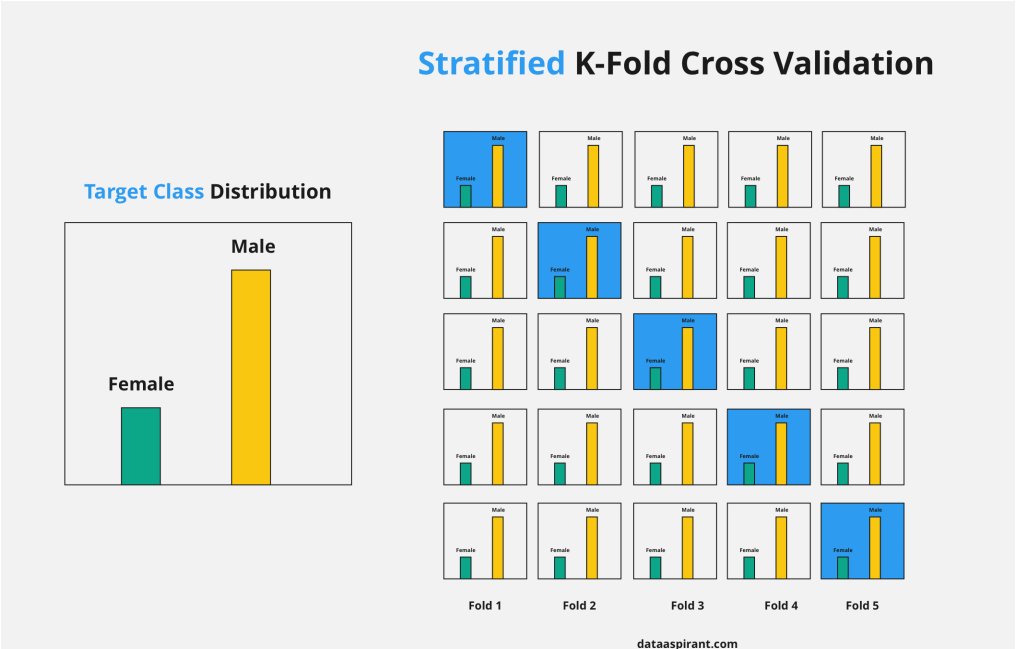


Advantages and Limitations

Advantages	Limitations
—Utilizes all available data for training and validation.	—Computationally expensive for large datasets.
—Provides an unbiased estimate of model performance.	—High variance in model performance estimation.
—Low bias as each model is trained on almost all data points.	—Not suitable for large sample sizes due to limited improvement over other methods.

Stratified Cross-Validation

Stratified cross-validation is a variation of k-fold cross-validation where each fold or partition of the data preserves the proportion of samples from each class or category. This ensures that the distribution of classes is consistent across all folds, making it particularly useful for classification tasks with imbalanced class distributions.



Advantages

- Preserves Class Distribution where each fold contains a representative sample of all classes,
- By preserving the class distribution in each fold, stratified cross-validation provides a more accurate estimate of model performance, leading to better generalization to unseen data.
- Helps to reduce bias in model evaluation,
- Improved Model Evaluation: Provides a more reliable assessment of model performance,

Advantages

- Preserves Class Distribution where each fold contains a representative sample of all classes,
- By preserving the class distribution in each fold, stratified cross-validation provides a more accurate estimate of model performance, leading to better generalization to unseen data.
- Helps to reduce bias in model evaluation,
- Improved Model Evaluation: Provides a more reliable assessment of model performance,

Advantages

- Preserves Class Distribution where each fold contains a representative sample of all classes,
- By preserving the class distribution in each fold, stratified cross-validation provides a more accurate estimate of model performance, leading to better generalization to unseen data.
- Helps to reduce bias in model evaluation,
- Improved Model Evaluation: Provides a more reliable assessment of model performance,

Disadvantages

- Complex Implementation, particularly when dealing with large and multi-class datasets.
- Dependency on Class Labels: Relies on accurate class labels for stratification, which may be challenging to obtain in certain real-world datasets or may introduce biases if the labels are noisy or inaccurate.
- In some cases, stratified cross-validation may lead to overfitting, particularly if the class distribution in the dataset is highly skewed or if the number of samples per class is very small.
- While highly beneficial for classification tasks, stratified cross-validation is not directly applicable to regression problems, where the target variable is continuous rather than categorical.

Disadvantages

- Complex Implementation, particularly when dealing with large and multi-class datasets.
- Dependency on Class Labels: Relies on accurate class labels for stratification, which may be challenging to obtain in certain real-world datasets or may introduce biases if the labels are noisy or inaccurate.
- In some cases, stratified cross-validation may lead to overfitting, particularly if the class distribution in the dataset is highly skewed or if the number of samples per class is very small.
- While highly beneficial for classification tasks, stratified cross-validation is not directly applicable to regression problems, where the target variable is continuous rather than categorical.

Disadvantages

- Complex Implementation, particularly when dealing with large and multi-class datasets.
- Dependency on Class Labels: Relies on accurate class labels for stratification, which may be challenging to obtain in certain real-world datasets or may introduce biases if the labels are noisy or inaccurate.
- In some cases, stratified cross-validation may lead to overfitting, particularly if the class distribution in the dataset is highly skewed or if the number of samples per class is very small.
- While highly beneficial for classification tasks, stratified cross-validation is not directly applicable to regression problems, where the target variable is continuous rather than categorical.

Disadvantages

- Complex Implementation, particularly when dealing with large and multi-class datasets.
- Dependency on Class Labels: Relies on accurate class labels for stratification, which may be challenging to obtain in certain real-world datasets or may introduce biases if the labels are noisy or inaccurate.
- In some cases, stratified cross-validation may lead to overfitting, particularly if the class distribution in the dataset is highly skewed or if the number of samples per class is very small.
- While highly beneficial for classification tasks, stratified cross-validation is not directly applicable to regression problems, where the target variable is continuous rather than categorical.

Outline

1. Model Selection
2. Cross-validation
   - K-Fold Cross-Validation
   - Leave-One-Out Cross-Validation (LOOCV)
   - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research

Lab 1: Social_Network_Ads (Logistic Regression)

Objective

Build a logistic regression classifier to predict whether a user purchases a product (`Purchased` $\in \{0, 1\}$) using two predictors: `Age` and `EstimatedSalary`.

The dataset contains user profiles from a social network advertisement campaign. Each row corresponds to one user.

Instructions

- Open a Python notebook (Jupyter or Google Colab).
- Create one section per step of the lab (Step 1 till 5).
- For each step: include your code, the requested plots/tables, and short answers to the questions.

Deadline: March 24

Step 1: Load & Inspect the Dataset

- Load the CSV file and inspect the table structure
- Identify predictors (`Age`, `EstimatedSalary`) and target (`Purchased`)
- Check missing values and class balance

Questions

- How many observations and variables?
- Is the target balanced?
- Any missing values?

Python hints

`pd.read_csv`, `df.head()`, `df.info()`, `value_counts()`, `isna().sum()`

Step 1: Load & Inspect the Dataset

- Load the CSV file and inspect the table structure
- Identify predictors (`Age`, `EstimatedSalary`) and target (`Purchased`)
- Check missing values and class balance

Questions

- How many observations and variables?
- Is the target balanced?
- Any missing values?

Python hints

`pd.read_csv`, `df.head()`, `df.info()`, `value_counts()`, `isna().sum()`

Step 2: Visualize the Classes (Separability?)

- Create a scatter plot in feature space: `Age` (x-axis) vs `EstimatedSalary` (y-axis)
- Color points by `Purchased` (0/1)

Questions

- Do the classes appear (approximately) linearly separable?
- What would a linear decision boundary look like here?

Python hints

- `import matplotlib.pyplot as plt`
- `plt.scatter(x, y, c=labels)`
- Color options such as `cmap="bwr"` and transparency `alpha=0.7`
- Axis labels with `plt.xlabel()` and `plt.ylabel()`

Step 3: Fit Logistic Regression (Train/Test)

- Split data into training and test sets
- Fit logistic regression on (`Age`, `EstimatedSalary`)
- Extract coefficients and interpret signs

Questions

- What are the signs of the coefficients?
- Which predictor increases the odds of purchase?

Python hints

- Split the data: `train_test_split` from `sklearn.model_selection`
- Fit the model: `LogisticRegression` from `sklearn.linear_model`
- Access coefficients: `clf.coef_` and `clf.intercept_`

Step 4: Predictions and Model Evaluation

- Use the trained model to predict:
  - class labels (0/1)
  - predicted probabilities
- Evaluate how well the model performs on the test set

Questions

- What is the difference between predicted probabilities and predicted classes?
- How accurate is the model on the test set?
- Is accuracy alone sufficient to evaluate performance?

Python hints

- `clf.predict(X_test)`
- `clf.predict_proba(X_test)`
- `accuracy_score` from `sklearn.metrics`

Step 5: Effect of the Classification Threshold

Logistic regression outputs probabilities in $[0, 1]$. A threshold τ is used to convert probabilities into class labels; We usually fix $\tau = 0.5$, but this is not always optimal

- Recompute confusion matrices for different values of τ

Questions

- What happens if the threshold is lowered (e.g. $\tau = 0.3$)?
- What happens if the threshold is increased (e.g. $\tau = 0.7$)?
- Which types of errors are affected (false positives vs false negatives)?

Python hints

- `clf.predict_proba(X_test)[:,1]`
- Convert to labels: `(proba >= tau).astype(int)`

Step 6: Regularization

Logistic regression estimates coefficients by maximizing the likelihood. However, this can lead to overfitting when:

- predictors are strongly correlated,
- the dataset is small, or we include many features.

Regularized logistic regression maximizes a penalized log-likelihood:

$$\hat{\beta} = \arg\max_{\beta} \Big(\ell(\beta) - \lambda \, \mathscr{P}(\beta) \Big)$$

- $\ell(\beta)$: log-likelihood (fit to the data)
- $\mathscr{P}(\beta)$: penalty on model complexity

Penalty terms

- L2 (Ridge): $\mathscr{P}_{L2}(\beta) = \sum_j \beta_j^2$
- L1 (Lasso): $\mathscr{P}_{L1}(\beta) = \sum_j |\beta_j|$

Step 6: Regularization

- L2 regularization (Ridge)
  - Penalizes the square of coefficients
  - Shrinks coefficients smoothly toward zero
  - Usually keeps all predictors
- L1 regularization (Lasso)
  - Penalizes the absolute value of coefficients
  - Can set some coefficients exactly to zero
  - Performs implicit feature selection

Intuition

- L2:Use all variables, but more cautiously.
- L1: Use only the most important variables.

Questions

- Which regularization would you choose if interpretability is important? Which one would you expect to be more stable?

Step 6: Regularization

In `scikit-learn`, regularization is controlled by the parameter `C`.

- **Large** $C \Rightarrow$ weak regularization (model fits data closely)
- **Small** $C \Rightarrow$ strong regularization (coefficients shrink)

Questions

- Try several values of `C` (e.g. 10, 1, 0.1, 0.01).
- How do the coefficients change as C decreases?
- How does test accuracy change?

Python hints

- `LogisticRegression(penalty="l2", C=C)`
- `LogisticRegression(penalty="l1", solver="liblinear", C=C)`

Step 7: K-Fold Cross-Validation

- Apply K-fold cross validation (e.g. $K = 5$)
- Report average performance across folds

Questions

- How does K-fold CV differ from a single train/test split?
- Why does K-fold reduce variance of the estimate?
- What happens if K is too small or too large? (A small plot of performance as a function of K is highly encouraged.)

Python hints

- `from sklearn.model_selection import KFold, cross_val_score`
- `kf = KFold(n_splits=5, shuffle=True)`
- `scores = cross_val_score(clf, X, y, cv=kf);scores.mean(), scores.std()`

Step 8: Leave-One-Out Cross-Validation (LOOCV)

- Apply LOOCV and report performance average on test out sample.

Questions

- Why can LOOCV have high variance?
- When would LOOCV be impractical?

Python hints

- `from sklearn.model_selection import LeaveOneOut`
- `loo = LeaveOneOut()`
- `scores = cross_val_score(clf, X, y, cv=loo)`

Step 9: Stratified Cross-Validation

- Apply the cv scheme on the Social$_N$*etwork*$_A$*dsdataset*.

Questions

- Is stratification highly important in this setting? Explain.
- Would stratification matter in regression?

Python hints

- `from sklearn.model_selection import StratifiedKFold`
- `skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)`
- `scores = cross_val_score(clf, X, y, cv=skf)`

Outline

1. Model Selection
2. Cross-validation
   - K-Fold Cross-Validation
   - Leave-One-Out Cross-Validation (LOOCV)
   - Stratified Cross-Validation
3. Lab: Classifcation Methods and CVs- continued
4. Research